SOFTWARE ENGINEERING LAB REPORT

Design Analysis Report

LAB 9 | Case Study: Food Delivery Application

Project Overview

The Food Delivery Application allows customers to discover restaurants, browse menus, place and pay for orders, track delivery, and provide feedback. Restaurants manage menus and incoming orders, while delivery partners accept assignments and update delivery status. An administrator supervises users, restaurants, payments, and operational issues.

Submitted by

Team Member	Roll Number
Deepak Singh	202401100200123
Deepak Maurya	202401100200121
Deepak Nimesh	202401100100122
Deependra Ahirwar	202401100200124

Department of Computer Science & Engineering

1. Introduction

This report analyses the proposed design of a Food Delivery Application that connects customers, restaurants, delivery partners, and payment services. It focuses on functional scope, architecture, data design, security, quality attributes, and risks.

2. Problem Statement

Customers need a convenient way to discover food, compare prices, place orders, pay securely, and track delivery. Restaurants need a digital channel to manage menus and incoming orders. Delivery partners need clear assignments and status updates. The system coordinates these actors while maintaining correct order and payment state.

3. Functional Requirements

• Register/login users and support role-based access for customers, restaurants, riders, and administrators.

• Browse/search restaurants, categories, menu items, prices, and availability.

• Maintain cart, validate delivery address, calculate totals, taxes, delivery fees, and discounts.

• Place, pay for, cancel where eligible, and track orders.

• Allow restaurants to accept/reject orders and update preparation status.

• Assign delivery partners and expose pickup/delivery progress.

• Send notifications, collect ratings/reviews, and provide administrative reports.

4. Non-Functional Requirements

Attribute	Design Goal
Performance	Typical menu and order APIs should respond quickly under expected load; cache safe read-heavy menu data.
Availability	Handle temporary failures with timeouts, retries, health checks, and graceful degradation.
Security	TLS, secure password hashing, least privilege, role checks, input validation, and secret management.
Reliability	Use transactional order updates, idempotent payment callbacks, audit logs, and recoverable background jobs.
Scalability	Separate stateless services and allow horizontal scaling; queue asynchronous notifications.
Usability	Clear checkout, fees, order status, accessible controls, and useful error messages.
Maintainability	Modular code, documented APIs, automated tests, and centralized logging/monitoring.

5. Proposed Architecture

A layered architecture is suitable: presentation layer (customer, restaurant, rider interfaces), application/API layer (authentication, menu, order, payment, delivery, notification), data layer (relational/document database and cache), and external integrations (payment provider, maps, messaging). A modular monolith is a reasonable starting point for a student project; high-traffic domains can later be extracted into independent services.

6. Core Data Model

Entity	Important Attributes / Relationships
User	user_id, name, email/phone, password_hash, role, created_at

Entity	Important Attributes / Relationships
Restaurant	restaurant_id, owner_id, name, address, opening_status, rating
MenuItem	item_id, restaurant_id, name, price, availability, category
Order	order_id, customer_id, restaurant_id, address_id, status, subtotal, fees, total, created_at
OrderItem	order_id, item_id, quantity, unit_price_snapshot
Payment	payment_id, order_id, provider_reference, amount, status, refund_status
Delivery	delivery_id, order_id, rider_id, pickup_time, delivered_time, status
Review	review_id, order_id, customer_id, rating, comment

7. Key Design Decisions and Rationale

• Snapshot item price and name in order lines so historical receipts remain correct after menu edits.

• Represent order status with a controlled state machine to prevent invalid transitions.

• Keep payment processing external and store provider references rather than raw card details.

• Use server-side total calculation; never trust prices or discounts submitted by the client.

• Use asynchronous notifications so messaging outages do not block order creation.

• Record timestamps and status history for customer support and operational analytics.

8. Security and Risk Analysis

Risk	Mitigation
Unauthorized access to orders	Check ownership/role on every protected request; do not rely on UI hiding.
Duplicate payment or order	Use idempotency keys, unique provider references, and safe webhook processing.
Stale menu availability	Revalidate item availability and price at checkout.
Location privacy exposure	Request only necessary permissions, limit retention, and restrict location access.
Service or provider outage	Timeouts, retry with backoff, circuit breakers, reconciliation jobs, and clear status messaging.
Data loss or corruption	Backups, migration discipline, transaction boundaries, and tested restore procedures.

9. Testing Strategy

• Unit tests: price calculation, discount rules, role checks, and order state transitions.

• Integration tests: database persistence, payment sandbox callbacks, maps adapter, and notifications.

• System tests: complete checkout-to-delivery workflow and cancellation/refund paths.

• Security tests: authentication, authorization, validation, rate limiting, and secret exposure checks.

• Performance tests: menu search, checkout bursts, and order-status polling.

10. Conclusion

A modular, state-driven design provides a maintainable foundation for the Food Delivery Application. The highest-priority engineering concerns are payment correctness, authorization, order-state consistency, and

dependable communication among restaurants, riders, and customers.